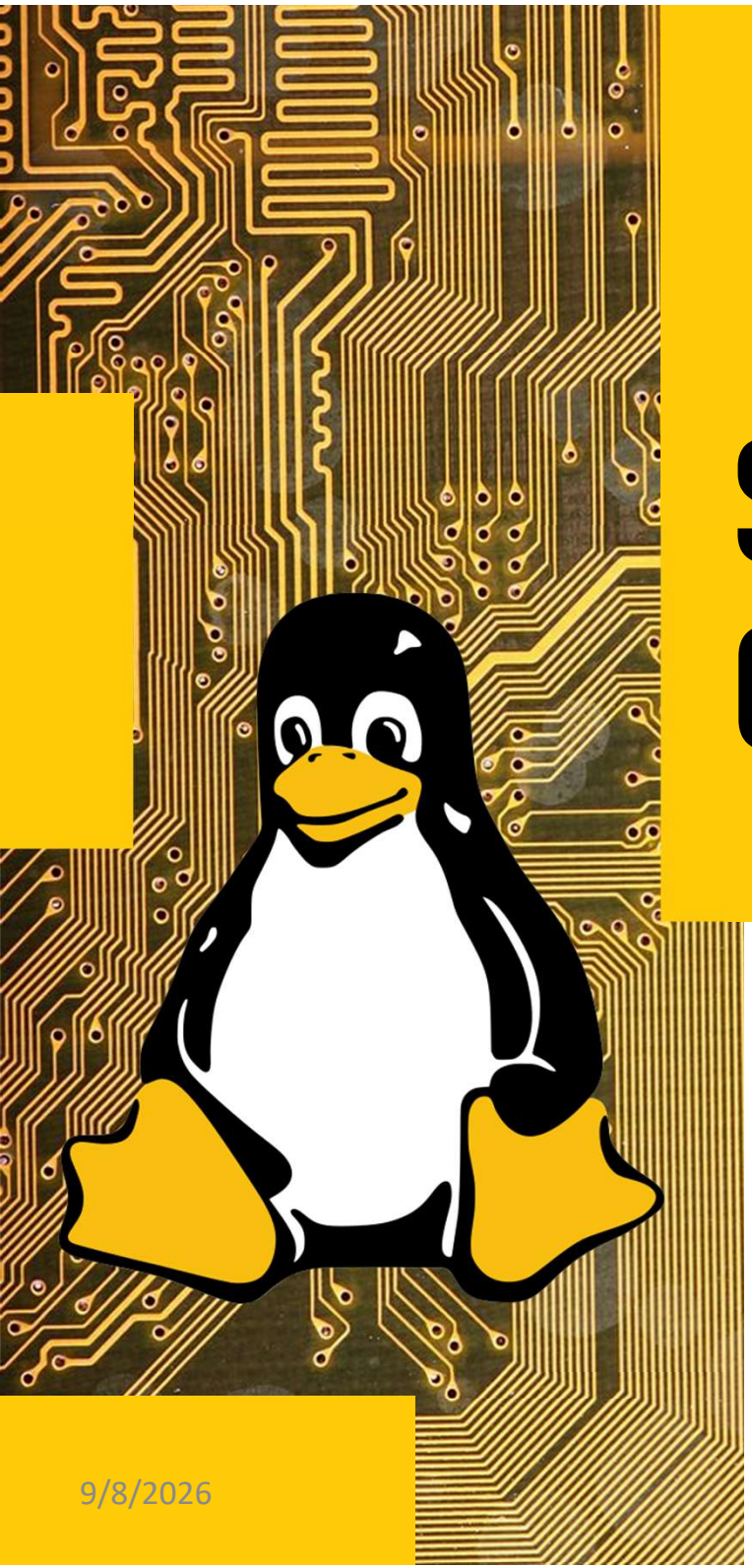


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Operating System Mechanics: Process Control and Signals

INSTRUCTOR: Jalolliddin Yusupov

Process Management

The Core Idea of fork(): Creates a new process that is a copy of the parent.

Both processes continue execution from the **exact same instruction immediately following the fork() call**. The child does *not* start from the top of main(); its Program Counter is copied from the parent at the moment of the fork.

Reading a Fork Program: The Three Rules

- **Rule 1: The Starting Line.** The child starts executing *at the return of* `fork()`, not at `main()`.
- **Rule 2: The Photocopy.** The child gets a complete *copy* of the parent's memory: variables, open file descriptors, and the instruction pointer. Changes in one do not affect the other.
- **Rule 3: The Dual Path.** Any code after `fork()` runs in *both* processes unless you branch based on the return value.

wait() & waitpid(): Reaping Child Processes

is a special function a program uses to ask the operating system to do something it can't do on its own—like waiting for another process to finish

What are they? System calls that allow a parent process to pause (or poll) and retrieve the *exit status* of a terminated (or stopped) child process. This is how a parent "collects the death certificate" and prevents the child from becoming a **zombie**.

```
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

is a process that has finished running, but its parent hasn't collected its exit status yet

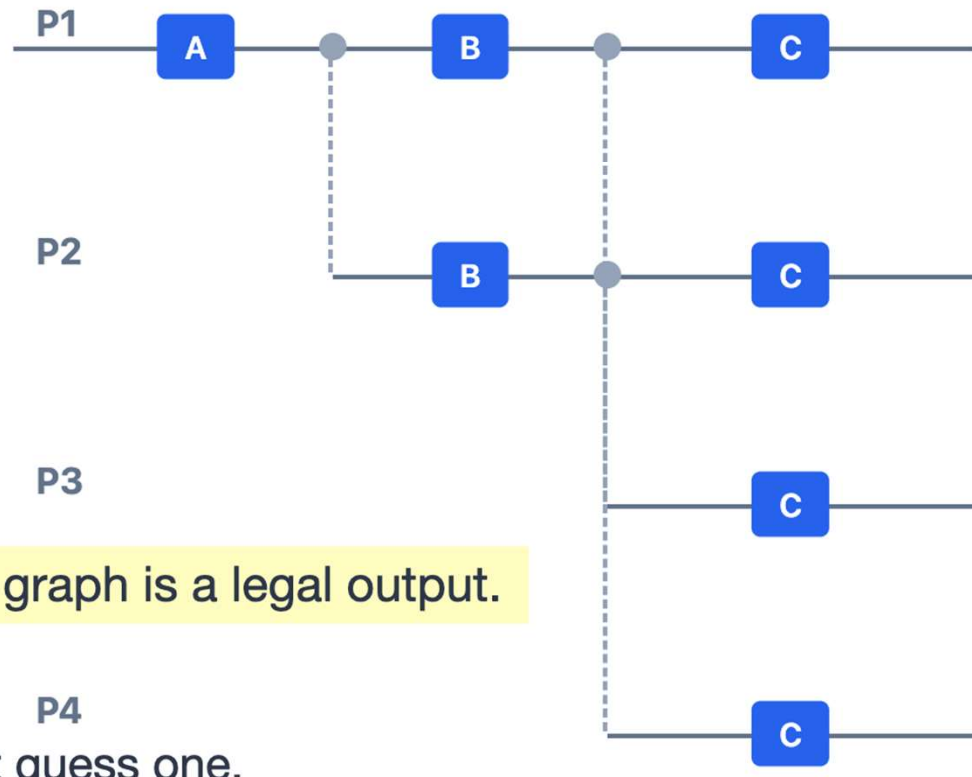
- `wait()` blocks until **any** child terminates, then returns that child's PID.
- `waitpid()` allows you to:
  - Wait for a **specific child** (`pid` argument).
  - Return immediately if no child has exited (`WNOHANG`).
  - Also report children that have been *stopped* (`WUNTRACED`).

Remember: If a parent never calls `wait`, its terminated children remain zombies until the parent exits (then they are adopted by PID 1). A well-behaved parent always reaps its children.

Process Graphs: The Method

The Notation:

- **Nodes:** Key operations (`printf` , `fork` , `wait` , `exit`).
- **Horizontal Lines:** A single process's timeline.
- **Vertical Arrows:** A `fork()` call.
- **Dashed Edges:** A `wait()` call, forcing an order.

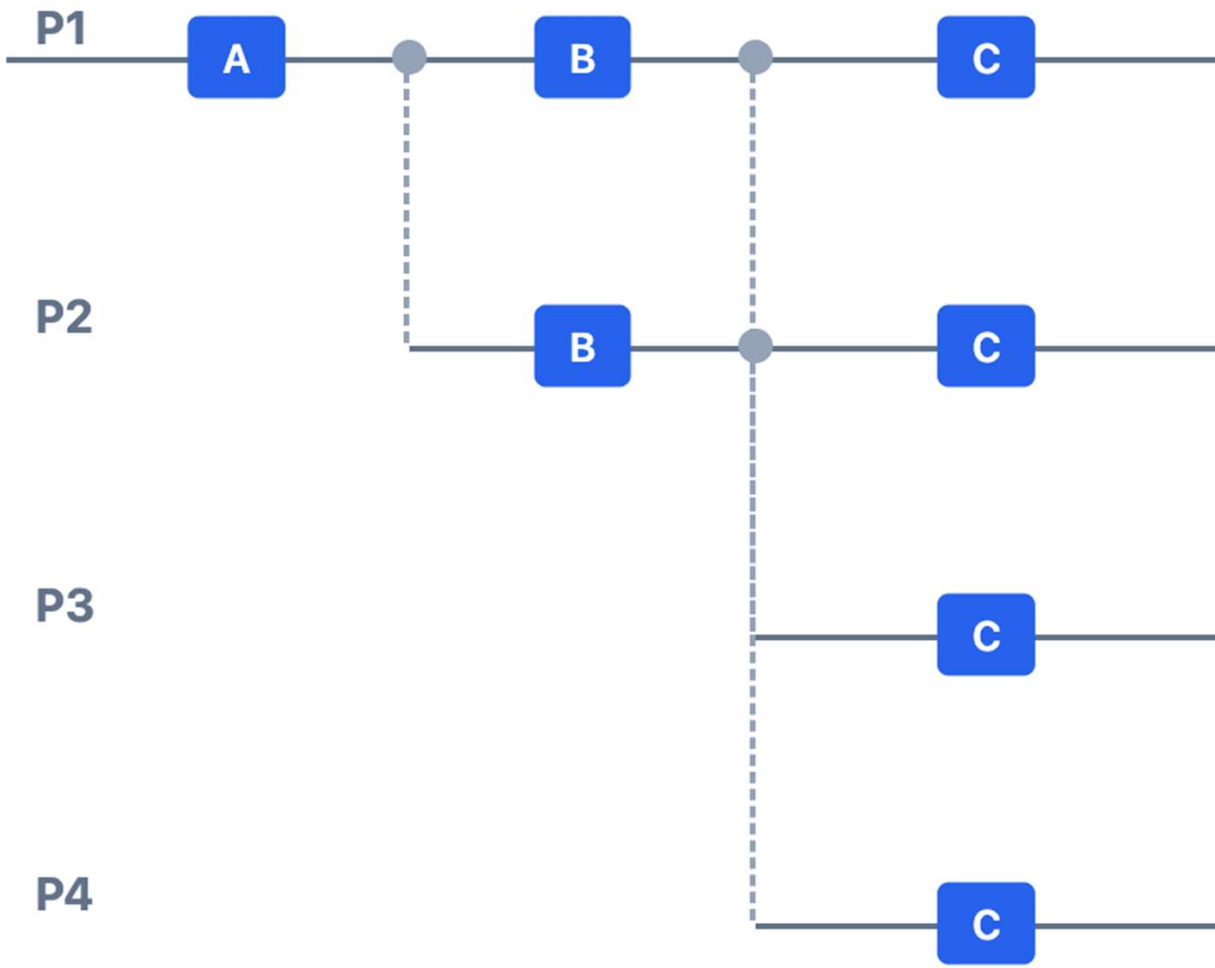


The Golden Rule: Every topological ordering of the graph is a legal output.

Exam Skill: You must *list all* possible outputs, not just guess one.

Program A: Two Forks, Straight Line

```
// fork_graph_a.c
write(1, "A\n", 2);
fork();
write(1, "B\n", 2);
fork();
write(1, "C\n", 2);
```



The Analysis:

- Total processes: **4**.
- Output Counts: A prints once, B prints twice, C prints four times. Total lines: **7**.

Program B: Fork in a Loop

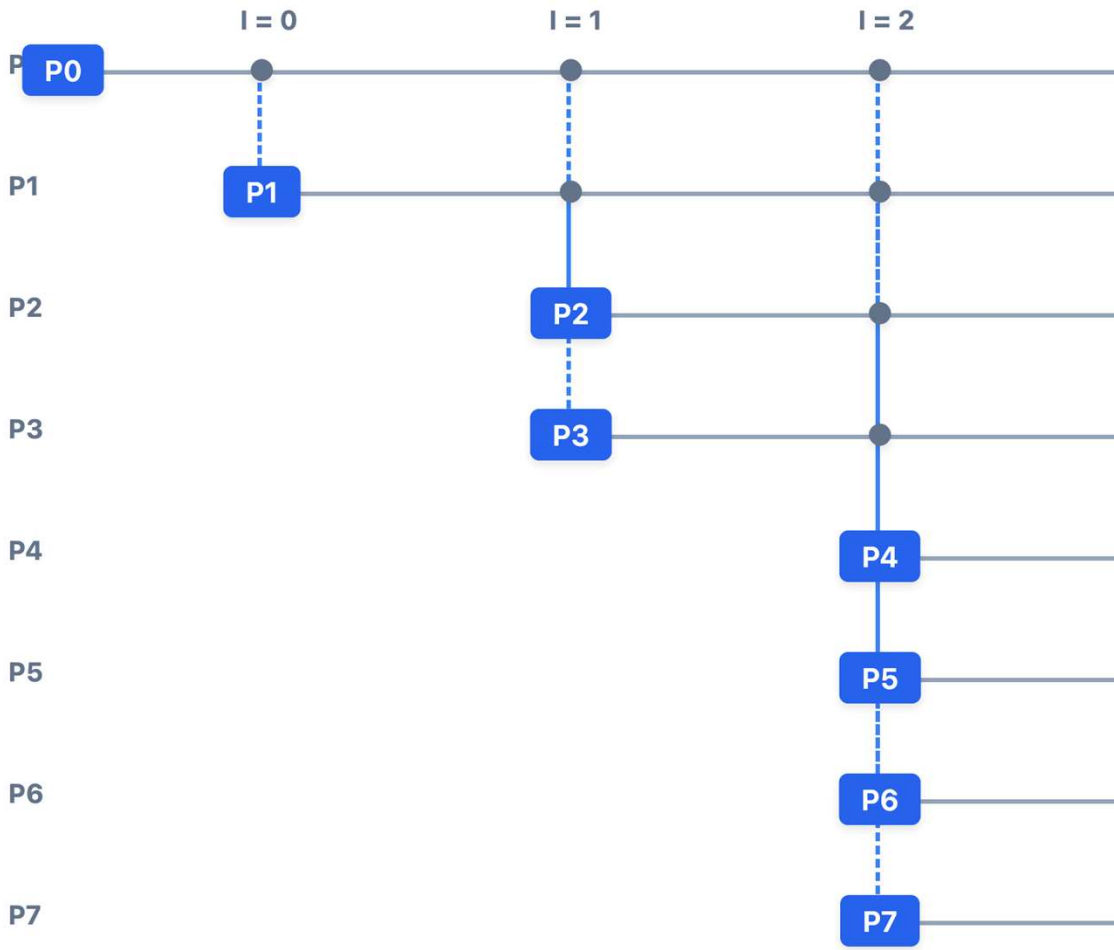
The Setup: A loop that runs `fork()` three times.

```
for (int i = 0; i < 3; i++) {
    fork();
}
```

```
--- Iteration i = 0 ---
P0 forks -> created P1
Active processes: 2 (2^1)

--- Iteration i = 1 ---
P0 forks -> created P2
P1 forks -> created P3
Active processes: 4 (2^2)

--- Iteration i = 2 ---
P0 forks -> created P4
P1 forks -> created P5
P2 forks -> created P6
P3 forks -> created P7
Active processes: 8 (2^3)
```



Program C: wait() Buys Order

Two runnable processes print in whatever order the scheduler likes. `wait()` takes that choice away from it — but read carefully **whose** order you bought.

EXAMPLE 1 — ONE CHILD, ONE WAIT

```c
if (fork() == 0) {
    printf("child:  I run first, always\n");
    exit(0);
}
wait(NULL);           // the constraint
printf("parent: I run second, always\n");
```

One name, in case it is new

`wait(NULL)` — the argument is where the child's exit status would be written. Passing `NULL` says *"I only want the ordering, not the status."* You already know the other half: the child ends with `exit(0)`, one of the three ways a process dies.

```
$ ./fork_graph_c 1   (three runs)
child:  I run first, always
parent: I run second, always
   identical every single run
```

The parent is **blocked** inside `wait()` until the child dies. There is only one legal order left, so the scheduler has nothing to decide.

EXAMPLE 2 — TWO CHILDREN, ONE WAIT

```c
for (int i = 0; i < 2; i++)
    if ((kid[i] = fork()) == 0) {
        if (i == 0) sleep(1);  // the first-born dawdles
        printf("child (number %d)\n", i);
        exit(0);
    }
first = wait(NULL);    // which one comes back?
```

```
$ ./fork_graph_c 2
child  (number 1)
parent: forked 0 first, reaped child number 1 first
child  (number 0)
```

The habit that is not a guarantee

Delete that `sleep` and the first-born wins **20 runs out of 20** — which is how a bug like this survives your testing.

`wait()` returns **whoever finishes**, not whoever was created. Want a particular child? Name it: `waitpid(kid[0], &st, 0)`.

wait() buys one ordering constraint per call — between you and one child. It buys nothing between the children.

How a Process Dies: Three Ways

1. `return` from `main` (= `exit`).
2. `exit(n)`
3. **Killed by a signal.**

Next of Kin

['nekst ōv 'kin]

The closest living relative of a deceased person, such as that person's spouse, siblings, or parents.

The exit status (0–255) is a **death certificate**; the kernel keeps it until next of kin collects it.

Zombies

A child that has exited but whose parent has not `wait`ed is a **zombie**



Orphans

Parent dies first → the child is **adopted** by PID 1

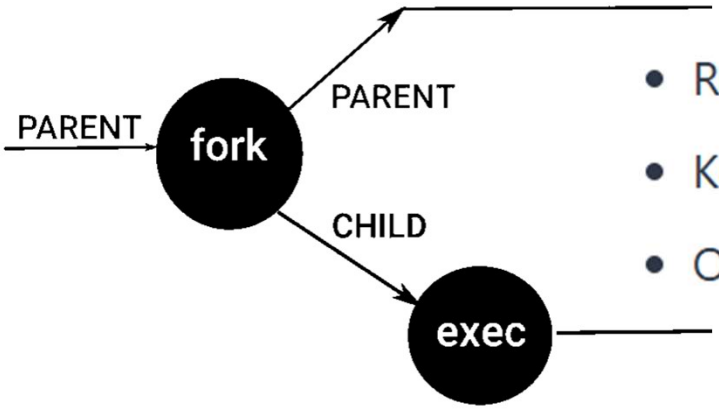
PID 1 - very first user-space process started by the Linux kernel during the system boot

`exec` : New Brain, Same Hands

Once a new process is created with `fork()`, it can choose to run something *entirely different*. That's where `exec()` comes in.

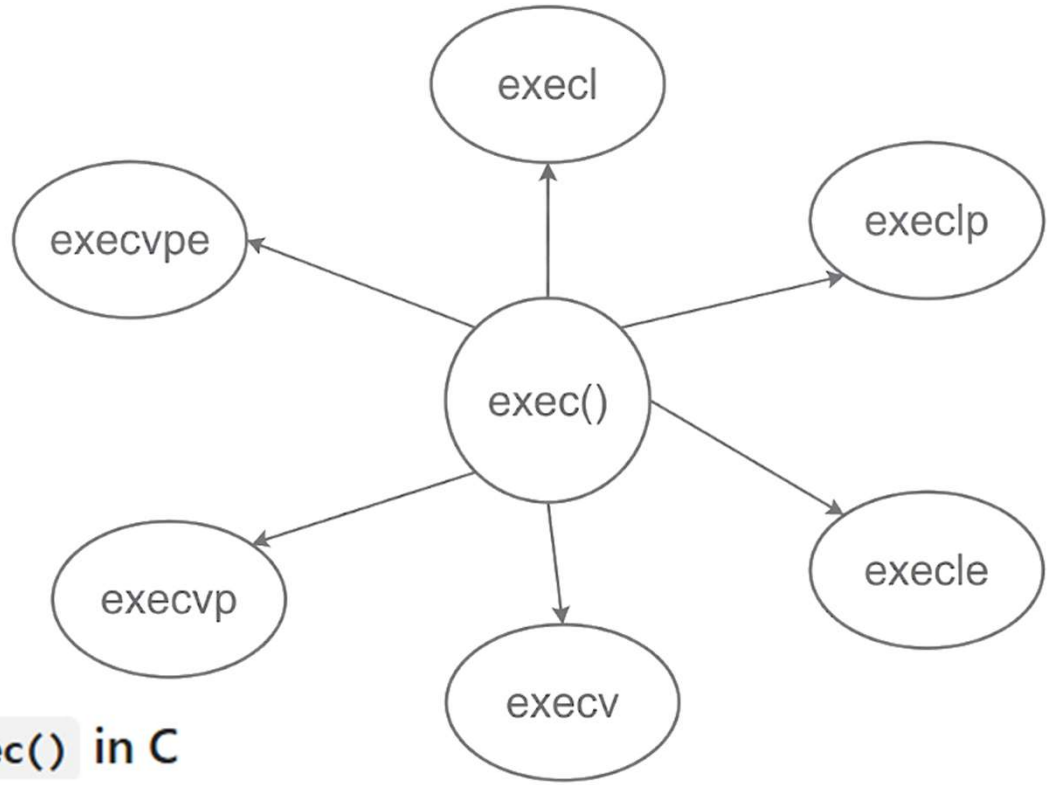
exec: The Missing Verb

`exec()` is a system call that replaces the current process with a new program. It does *not* create a new process, it transforms the existing one.



- Replaces the program image (code, data, heap, stack).
- Keeps the **PID, PPID, open files, cwd**.
- On success it **never returns** — the line after it is the failure path.

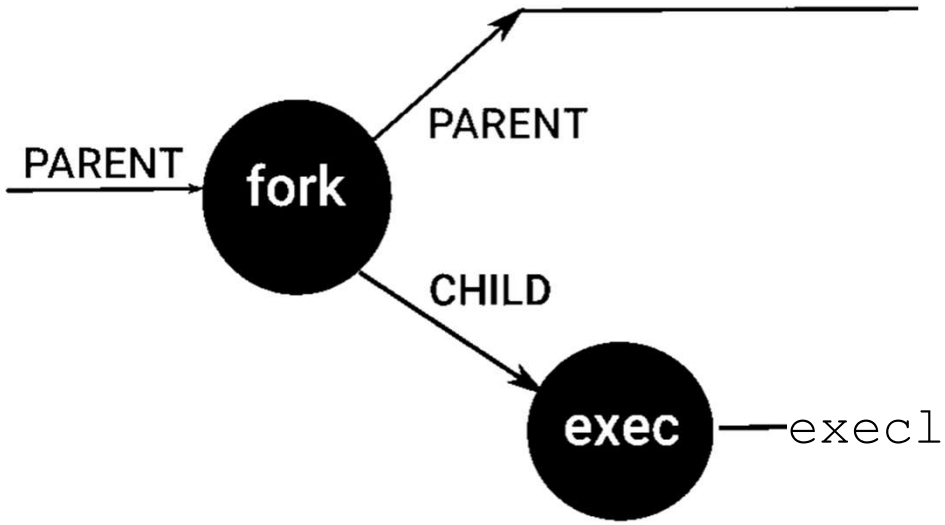
exec() Family System Calls



there is no function literally named `exec()` in C

execlp example

```c
#include <stdio.h>
#include <unistd.h> // Required for exec family functions
int main() {
    printf("1. I am the original C program. My PID is %d\n", getpid());
    printf("2. I am about to replace myself with the 'ls -l' command...\n");
    // The exec call
    // 1st arg: "ls"      -> The program to find in the PATH
    // 2nd arg: "ls"      -> argv[0] (The name of the program as it sees itself)
    // 3rd arg: "-l"      -> argv[1] (The first argument to pass to the program)
    // 4th arg: NULL      -> REQUIRED: Marks the end of the argument list
    execlp("ls", "ls", "-l", NULL);
    // We only reach this point if execlp FAILED (e.g., program not found)
    return 1;
}
```



execv (Vector / Array)

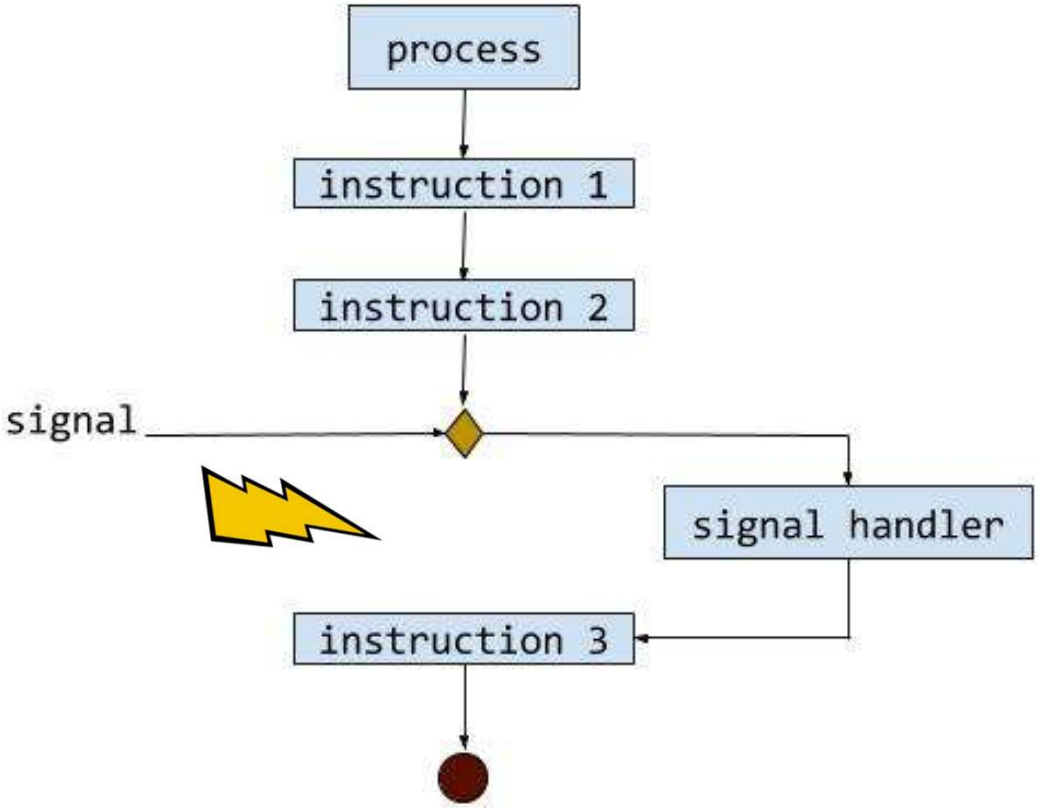
```
// 1. Build the arguments as an array (vector)
// MUST end with NULL
char *my_args[] = {"echo", "Hello", "from", "execv!", NULL};
// 2. Call execv (requires EXACT path, no 'p')
execv("/bin/echo", my_args);
```

execl (List, Exact Path)

```
// Call execl:
// 1st arg: exact path to the binary
// 2nd arg: argv[0] (name of the program)
// 3rd arg: argv[1] (the format string for date)
// 4th arg: NULL terminator
execl("/bin/date", "date", "+%T", NULL);
```

- **l** (list) – arguments passed as a list: `execl`, `execlp`, `execle`
- **v** (vector) – arguments passed as an array: `execv`, `execvp`, `execvpe`
- **p** (PATH) – search `PATH` for the program: `execlp`, `execvp`

SIGNALS, PROPERLY



02 / 04

The signal() Function

FUNCTION SIGNATURE

```
#include <signal.h>
typedef void (*sighandler_t)(int);
    sighandler_t     signal(int signum, sighandler_t handler);
                Returns: ptr to previous handler if OK, SIG_ERR on error (does not set errno)
```

THREE WAYS TO CHANGE A SIGNAL'S ACTION

HANDLER VALUE	EFFECT	
SIG_IGN	Signals of type *signum* are **ignored**.	`// Tell the OS to IGNORE SIGINT (Ctrl+C)` `signal(SIGINT, SIG_IGN);`
SIG_DFL	Action **reverts to the default** for *signum*.	`// Now that we handled it once, let's restore the DEFAULT behavior.` `// The next time the user presses Ctrl+C, the program will just die.`
&my_handler	Address of a **user-defined signal handler** — called whenever the process receives *signum*.	

Installing Signal Handlers

HOW HANDLERS ARE INVOKED

1. When a process catches signal *k*, the handler installed for *k* is called with a single `int` argument set to *k*.
2. This lets **one function handle multiple signal types** — branch on the argument to distinguish them.
3. **SIGINT example:** the shell sends `SIGINT` when the user types `Ctrl-C`. Default action: terminate process. A custom handler can catch it, print a message, then exit cleanly.

EXAMPLE — SIGINT HANDLER

sigint.c

```c
void handler(int sig) {
  printf("Caught SIGINT!\n");
  exit(0);
}

int main() {
  signal(SIGINT, handler);
  pause(); /* wait for signal */
  exit(0);
}
```

KEY TERMINOLOGY

- **I** **Installing** the handler — passing a function address to `signal()` to change the default action.
- **C** **Catching** the signal — the invocation of the handler when the signal arrives.
- **H** **Handling** the signal — the execution of the handler body.
- **R** **Return** — when the handler executes its `return` statement, control passes back to the interrupted instruction in main. (On some systems, interrupted system calls return with an error instead.)

```
yujboss@PositiveHPDesktop:~$ kill -2 12274
Caught SIGINT!
```

Signal Types: Catchable vs Uncatchable

Common Signals (Linux x86-64)

Signal	#	Default	Typical Source
`SIGINT`	2		Ctrl-C (terminal)
`SIGQUIT`	3		Ctrl-\
`SIGKILL`	9		`kill -9`
`SIGSEGV`	11		Invalid memory access
`SIGTERM`	15		`kill` (default)
`SIGSTOP`	19		`kill -STOP`
`SIGTSTP`	20		Ctrl-Z
`SIGCHLD`	17		Child stopped/exited

Terminate = turning off the computer. Done.

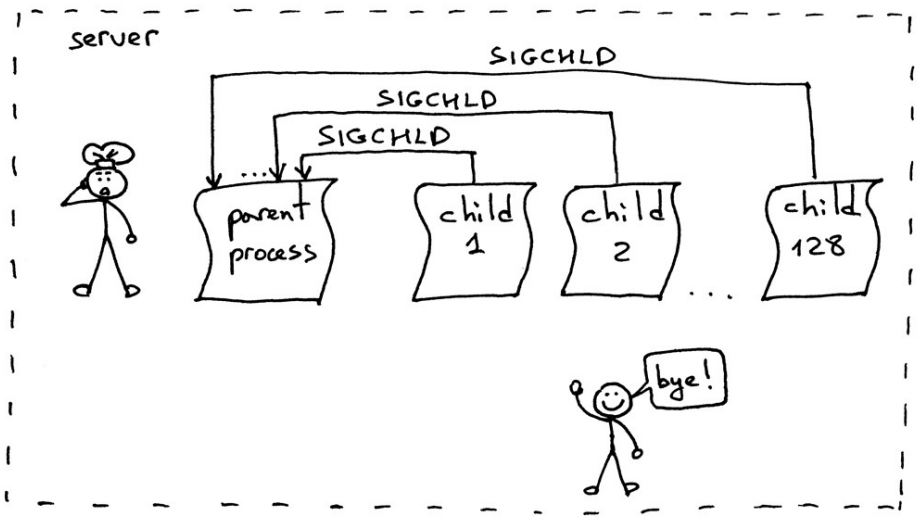
Core Dump = turning off the computer, but first taking a photo of the screen for debugging.

Stop = pressing the pause button. You can press play again later.

SIGCHLD: The Child Status Signal

What is it? The system sends `SIGCHLD` to a parent process whenever one of its children **exits** or **stops**. It's the system's way of saying, "One of your children needs attention — come collect its status."

- **Signal number:** 17 (on Linux x86-64)
- **Default action:** Ignore
- **Catchable?** Yes
- **Generated by:** The system, automatically




```c
void handle_sigchld(int sig) {
    /*
     * WHY A 'while' LOOP INSTEAD OF AN 'if' STATEMENT?
     * Standard POSIX signals are UNQUEUED. If 5 child processes die at the exact same
     * instant, the kernel might deliver only ONE SIGCHLD signal.
     * Using a 'while' loop forces waitpid() to run repeatedly in a loop until EVERY
     * finished child is cleaned up, preventing leftover "zombie" processes.
     */
    while (waitpid(-1, NULL, WNOHANG) > 0) {
        /*
        *   -1  (Target PID): "Wait for ANY child process" belonging to this parent.
        *   NULL  (Status Pointer): Ignore the child's exit status code.
        *   WNOHANG  (Options Flag: "Wait NO HANG") :Return immediately without blocking/freezing execution.
        * ------------------------------------------------------------
         ////// Child successfully reaped
    }
}
```

sigaction(): The Modern Way to Handle Signals

What is it? A system call that lets you **precisely** control how a signal is handled. It replaces the older, unreliable `signal()` function.

The Problem with signal()

- **Not portable:** Behavior varies between Unix versions (BSD vs System V).
- **Handler reset:** On some systems, the handler resets to default after one delivery.
- **No control over syscall restart:** Can't decide if interrupted calls should resume.
- **Race condition:** A signal can arrive before the handler is fully installed.

```c
void handle_sigint(int sig) {
    // safe work only: write, _exit, or set a flag
    got_sigint = 1;
}
int main() {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);       // no extra signals blocked during handler
    sa.sa_flags = SA_RESTART;       // automatically restart interrupted syscalls
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        exit(1);
    }
    // ... rest of program
}
```

"`signal()` is for reading old code; `sigaction()` is for writing new code."

sigaction(): The Modern Way to Handle Signals

```
#include <signal.h>

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

Uses the **struct sigaction** type to specify which signal handler to run and other options for how the signal should be handled

```
struct sigaction {
    void     (*sa_handler)(int);     // handler function or SIG_DFL/SIG_IGN
    void     (*sa_sigaction)(int, siginfo_t *, void *); // alternative handler
    sigset_t   sa_mask;              // signals to block during handler
    int        sa_flags;             // options (SA_RESTART, SA_SIGINFO, etc.)
    void     (*sa_restorer)(void);   // obsolete, not used
};
```

`SA_RESTART`	Automatically restart interrupted system calls (like `read`, `write`).
`SA_SIGINFO`	Use `sa_sigaction` handler, which receives extra info about the signal.



Questions?